

实验报告

课程名称: Design pattern and software architecture

学 院: 计算机科学与工程学院

专 业: Computer Science and 班 级: SE-1

Technology

姓 名: Muhammad Samudera 学 号:

年 月 日

山东科技大学教务处制

实 验 报 告

_____页

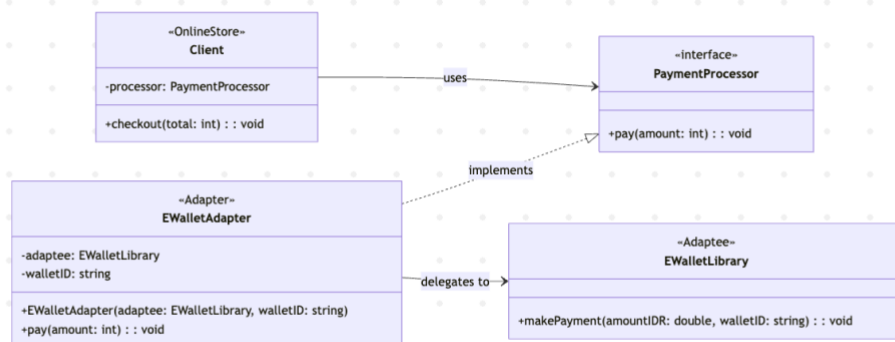
组 别	1	姓 名	Samudera	同组实验者	None
实验项目名称	Adapter Pattern				
教师评语	Good understanding of the pattern idea and implementation. The report is clear and written in proper English.				
实验成绩:			指导教师（签名）： 2026 3 23		
1. Objective The objective of this experiment is to implement the Adapter design pattern using an E-Wallet payment system example involving PaymentProcessor, EWalletLibrary, and EWalletAdapter. The goal of this pattern is to allow two incompatible interfaces to work together by introducing an intermediary adapter class that translates calls from one interface into the format expected by the other. In this example, the program has four key participants: • PaymentProcessor (Target Interface): defines the standard payment interface that the client expects, with a simple pay(int amount) method. • EWalletLibrary (Adaptee): an external library with an incompatible method signature makePayment(double amountIDR, String walletID) that cannot be modified. • EWalletAdapter (Adapter): bridges the gap between PaymentProcessor and EWalletLibrary by implementing the target interface and internally delegating calls to the adaptee. • Client (OnlineStore): uses PaymentProcessor to process payments without any knowledge of the underlying EWalletLibrary. The Adapter class is responsible for converting the client's request into a form that the adaptee understands, enabling seamless integration without modifying either the client or the external library. 2. Principle The Adapter pattern converts the interface of a class into another interface that clients expect. It lets classes work together that could not otherwise because of incompatible interfaces. Instead of					

modifying existing code to match a new interface, an adapter class is introduced to perform the translation.

Key benefits of this approach include:

- The client (OnlineStore) depends only on the PaymentProcessor interface, not on any specific payment library implementation.
- The external library EWalletLibrary remains untouched, preserving its original code and avoiding the risk of introducing bugs.
- New payment methods can be added by creating new adapter classes without modifying existing client code, adhering to the Open/Closed Principle.
- The adapter handles data conversion (int to double) and supplies additional parameters (walletID), hiding complexity from the client.

3. UML Diagram



4. Key Code (Java)

```
// TARGET INTERFACE – defines the interface the client expects

interface PaymentProcessor {

    void pay(int amount);

}
```

```

// ADAPTEE – external library with an incompatible interface

class EWalletLibrary {

    public void makePayment(double amountIDR, String walletID) {

        System.out.println("[EWalletLibrary] Processing payment...");

        System.out.println("  Wallet ID   : " + walletID);

        System.out.println("  Amount IDR : Rp " + String.format("%.0f", amountIDR));

        System.out.println("[EWalletLibrary] Payment successful!");

    }

}

// ADAPTER – bridges PaymentProcessor and EWalletLibrary

class EWalletAdapter implements PaymentProcessor {

    private EWalletLibrary adaptee;

    private String walletID;

    public EWalletAdapter(EWalletLibrary adaptee, String walletID) {

        this.adaptee = adaptee;

        this.walletID = walletID;

    }

    @Override

```

```

public void pay(int amount) {

    // Conversion: int → double (matches EWalletLibrary signature)

    double amountIDR = (double) amount;

    adaptee.makePayment(amountIDR, walletID);

}

}

// CLIENT – uses PaymentProcessor without knowing about EWalletLibrary

class Client {

    private PaymentProcessor processor;

    public Client(PaymentProcessor processor) {

        this.processor = processor;

    }

    public void checkout(int total) {

        System.out.println("[OnlineStore] Starting checkout with total: Rp "

            + String.format("%,d", total));

        processor.pay(total);

        System.out.println("[OnlineStore] Checkout completed.");

    }

}

```

```
// MAIN – demonstrates the Adapter pattern in action

EWalletLibrary eWalletLibrary = new EWalletLibrary();

PaymentProcessor adapter = new EWalletAdapter(eWalletLibrary, "WALLET-2025-XYZ");

Client onlineStore = new Client(adapter);


onlineStore.checkout(150000); // → EWalletLibrary.makePayment(150000.0, "WALLET-2025-XYZ")

onlineStore.checkout(850000); // → EWalletLibrary.makePayment(850000.0, "WALLET-2025-XYZ")

onlineStore.checkout(2500000); // → EWalletLibrary.makePayment(2500000.0, "WALLET-2025-XYZ")
```

5. Analysis

- **Advantages:** The Adapter pattern provides a clean mechanism for integrating incompatible interfaces without modifying existing code. In this program, the Client class interacts exclusively with the PaymentProcessor interface and has no direct dependency on EWalletLibrary. This means the external library can be replaced or upgraded independently, as long as a corresponding adapter is provided. The pattern also promotes the Single Responsibility Principle: the adapter class is solely responsible for interface translation, keeping the client logic and the library logic separate. Furthermore, the adapter encapsulates the data conversion from int to double and the injection of the walletID parameter, shielding the client from these implementation details.
- **Disadvantages:** The Adapter pattern introduces additional classes into the codebase, which can increase complexity when many adapters are required for different services. Each new external payment provider would need its own adapter class, which could lead to a proliferation of small wrapper classes. Additionally, the adapter adds a layer of indirection every call to pay() goes through the adapter before reaching the actual library — which, while negligible in terms of performance, can make debugging slightly harder since the call stack is deeper. If the adaptee's interface changes significantly, the adapter must also be updated, creating a maintenance dependency.

6. Conclusion

The Adapter pattern is well-suited for situations where an existing class needs to be used but its interface does not match what the client expects. In this program, EWalletLibrary represents a third-party payment library with a specific method signature (makePayment(double, String)), while the client code expects a simpler and standardized PaymentProcessor interface (pay(int)). The

EWalletAdapter bridges this gap by implementing PaymentProcessor and internally delegating calls to EWalletLibrary with the necessary parameter conversions.

The Adapter pattern is well-suited for situations where an existing class needs to be used but its interface does not match what the client expects. In this program, EWalletLibrary represents a third-party payment library with a specific method signature (makePayment(double, String)), while the client code expects a simpler and standardized PaymentProcessor interface (pay(int)). The EWalletAdapter bridges this gap by implementing PaymentProcessor and internally delegating calls to EWalletLibrary with the necessary parameter conversions.

The Adapter pattern is well-suited for situations where an existing class needs to be used but its interface does not match what the client expects. In this program, EWalletLibrary represents a third-party payment library with a specific method signature (makePayment(double, String)), while the client code expects a simpler and standardized PaymentProcessor interface (pay(int)). The EWalletAdapter bridges this gap by implementing PaymentProcessor and internally delegating calls to EWalletLibrary with the necessary parameter conversions.

This approach cleanly separates the concerns of interface compatibility from business logic. The client (OnlineStore) does not need to know how payments are processed internally; it simply calls pay() on its PaymentProcessor reference. The adapter handles all translation, including type conversion and supplying the wallet ID. This is confirmed by the program output, where each checkout call successfully triggers the EWalletLibrary's payment processing with the correct amount and wallet ID.

Overall, the Adapter pattern is a practical and widely-used structural pattern for achieving interoperability between incompatible interfaces. It preserves the integrity of existing code, promotes loose coupling, and makes the system easier to extend with new payment providers in the future. When applied in scenarios involving third-party library integration or legacy system migration, it serves as a fundamental and reliable building block in software architecture.

